

Name : P. Yaswanth

Reg No : 192524167

Experiment: Alpha-Beta Pruning Algorithm for Gaming

Aim

To implement the **Alpha-Beta Pruning Algorithm** using Python to optimize the Minimax algorithm for game-playing applications.

Objectives

- To understand the Alpha-Beta Pruning algorithm.
- To implement Alpha-Beta Pruning using Python.
- To reduce the number of nodes evaluated in the game tree.
- To determine the optimal move for the maximizing player.
- To understand the application of Alpha-Beta Pruning in Artificial Intelligence.

Algorithm

1. Create a game tree with terminal node values.
2. Initialize **alpha = $-\infty$** and **beta = $+\infty$** .
3. Start from the root node.
4. If it is the maximizing player's turn, update the alpha value.
5. If it is the minimizing player's turn, update the beta value.
6. If **alpha $\geq$ beta**, prune the remaining branches.
7. Continue until the optimal value is obtained.
8. Display the optimal value.

Python Program

```
import math
```

```
# Alpha-Beta Pruning Function
```

```
def alphabeta(depth, nodeIndex, maximizingPlayer, values, alpha, beta, maxDepth):
```

```
# Leaf node

if depth == maxDepth:
    return values[nodeIndex]

if maximizingPlayer:
    best = -math.inf

    for i in range(2):
        val = alphabeta(depth + 1,
                        nodeIndex * 2 + i,
                        False,
                        values,
                        alpha,
                        beta,
                        maxDepth)

        best = max(best, val)
        alpha = max(alpha, best)

    if beta <= alpha:
        break

    return best

else:
```

```
best = math.inf
```

```
for i in range(2):
```

```
    val = alphabeta(depth + 1,  
                    nodeIndex * 2 + i,  
                    True,  
                    values,  
                    alpha,  
                    beta,  
                    maxDepth)
```

```
    best = min(best, val)
```

```
    beta = min(beta, best)
```

```
    if beta <= alpha:
```

```
        break
```

```
return best
```

```
# Driver Code
```

```
values = [3, 5, 6, 9, 1, 2, 0, -1]
```

```
maxDepth = 3
```

```
result = alphabeta(  
    0,  
    0,
```

```
True,  
values,  
-math.inf,  
math.inf,  
maxDepth  
)  
  
print("The optimal value is:", result)
```

Sample Output

The optimal value is: 5

Out Put

```
import math  
  
# Alpha-Beta Pruning Function  
def alphabeta(depth, nodeIndex, maximizingPlayer, values, alpha, beta, maxDepth):  
  
    # Leaf node  
    if depth == maxDepth:  
        return values[nodeIndex]  
  
    if maximizingPlayer:  
        best = -math.inf  
  
        for i in range(2):  
            val = alphabeta(depth + 1,  
                             nodeIndex * 2 + i,  
                             False,  
                             values,  
                             alpha,  
                             beta,  
                             maxDepth)  
  
            best = max(best, val)  
            alpha = max(alpha, best)  
  
            if beta <= alpha:  
                break  
  
        return best  
    else:
```

The optimal value is: 5
=== Code Execution Successful ===

Result

The Alpha-Beta Pruning algorithm was successfully implemented using Python. The program evaluated the game tree efficiently and determined the optimal value while pruning unnecessary branches.

Conclusion

The experiment demonstrated the implementation of the **Alpha-Beta Pruning** algorithm, an optimization of the Minimax algorithm. By pruning branches that cannot affect the final decision, the algorithm reduces the number of nodes evaluated while still producing the optimal result. This improves the efficiency of game-playing AI and is widely used in games such as Chess, Checkers, Tic-Tac-Toe, and other strategic decision-making applications.